AlphaAudit

Can an AI write a trading strategy that is real, and can anything tell us in advance?

Ashwin Jain

Department of Computer Science and Engineering
Motilal Nehru National Institute of Technology Allahabad
Prayagraj, India

ashwinaadi2007@gmail.com

A benchmark for measuring whether machine-generated quantitative research survives transaction costs, multiple testing and a never-touched holdout — and whether an auditor can tell which candidates will, before the holdout is opened.

Code, corpus, fixtures and evaluation protocol:
github.com/Ashwin-aadi/Trivijaya-Quant-Research
Programme overview and results: ashwin-aadi.github.io/Trivijaya-Quant-Research

8 August 2026

Abstract

A language model that writes trading strategies is a machine for manufacturing plausible backtests. The two defects that make such backtests worthless — code that secretly reads the future, and the near-certainty that something looks brilliant if you try enough things — are precisely the two an ordinary evaluation cannot see. As models become better programmers, the backtests become more plausible, not more true, and the problem gets worse rather than better.

We release AlphaAudit: an environment, a protocol and a corpus for measuring whether machine-written trading research is real. It comprises a survivorship-free point-in-time universe of Indian equities, a transaction-cost model built to the Indian statutory schedule and verified against a broker's public calculator, deliberately-cheating and honest fixture suites, a hash-chained trial ledger the generator cannot write to, a frozen one-shot evaluation protocol, and 2,495 audited machine-written candidates.

What we measured. Four generators — one local 7B model and three frontier models — and 6 generation methods, all issued the same frozen task specification and all run through the same unchanged funnel. The question the programme asks is not which model writes the best code. It is whether making the AI better makes the *research* better.

The headline result is a separation. On the local 7B, the binding constraint is not cheating but competence: every draw returned parseable, interface-conforming code, and only 14.6% produced a strategy that both ran on real data and took a position. Frontier models remove that constraint completely — 60 of 60 candidates executed and traded, a rate of 100% against 14.6%. **They do not remove the second constraint at all.** Not one of the 60 frontier candidates attained a deflated Sharpe probability above 0.95 at its own honest trial count, on development or on holdout; 11 of them reverse the sign of their Sharpe once Indian costs are charged; and across ten independently generated corpora and 70 auditor configurations, 0 beat random rejection at matched coverage out of sample.

Better code did not become better evidence. We report that as the benchmark's first entry rather than as a result to be improved upon: the environment, the protocol and the fixtures are independent of whether our auditor worked, and a stronger auditor can contest the second claim against exactly the same apparatus.

Keywords: backtest overfitting • deflated Sharpe ratio • lookahead bias • selective prediction • LLM code generation • transaction costs • Indian equities

Key findings

Each of the six numbers below answers a question. Every one carries the sample it was measured on, and every one is tagged with the population it describes — because the distance between *"we observed this in our corpus"* and *"AI does this"* is where this literature usually goes wrong.

1. Does the AI cheat, or does it simply not work?

It does not work. All 1,550 local draws returned syntactically valid, interface-conforming Python with a stated rationale — a 100% success rate at looking correct. Only 631 (40.7%) survived contact with real data, and of those 406 (64.3%) never took a position. **14.6% of the corpus produced a strategy that both executes and trades.** An auditor built to catch cheating is guarding a door most candidates never reach.

2. Do frontier models fix that? [FRONTIER]

Completely. 60 of 60 candidates from three frontier models executed to completion and took a position: **100% against the local model's 14.6%.** The execution barrier that dominates the local corpus is an artefact of model capability, and it disappears when capability rises. Total static leak findings across all 60: 1.

3. Does better code become better evidence? [FRONTIER] [GENERATOR-INDEPENDENT]

No. Zero of 60 frontier candidates cleared a deflated Sharpe probability of 0.95 at its own trial count — 0 on development, 0 on the holdout — matching the local corpus's 0 of 631. Median holdout Sharpe is *worse* for every frontier arm than for the local reference (-1.149, -0.770, -1.149 against -0.616). **Frontier models solved the execution problem. They did not touch the evidence problem.**

4. Are transaction costs a haircut or a filter? [FRONTIER]

A filter. 15.6% of the local corpus's 225 tradeable strategies reverse the sign of their Sharpe once Indian costs are charged; none reverse the other way, as none can; 11 are bankrupted outright. 11 of 60 frontier candidates flip as well. The best surviving strategy in the corpus survives *because it barely trades*, at 2.8× annual turnover. **Cost survival selects for low turnover, not for skill.**

5. Does more elaborate prompting help? [GENERATION METHOD]

Not at equal compute. Of 5 scaffolded generation methods — chain-of-thought, planning, reflection, graph-of-thoughts and Monte Carlo tree search — **0 beat plain prompting once plain prompting was given the same token budget** and allowed to keep its best of k attempts. The scaffolding buys candidates, and the control buys the same number of candidates more cheaply.

6. Can the auditor tell in advance which ones are real? [GENERATOR-INDEPENDENT]

Not on any corpus we generated. Across 10 independently generated corpora — six generation methods, three frontier models, and the pooled local corpus — and all seven combinations of the three auditor layers, **0 of 70 configurations beat random rejection at matched coverage on out-of-sample data.** We publish the null rather than tune past it. It survived two subsequent corrections of our own pipeline.

One programme, three questions. Each paper answers the question the previous one raises, on one shared universe, one shared cost model and one shared backtest engine.

1	**AlphaAudit**	Can an AI produce a trading strategy that is real, rather than a plausible artefact of luck and leakage?
2	**RegimeStress**	If a strategy looks real, when does it break?
3	**FlowState**	If a strategy survives, how much capital can it absorb?

Code → validity → robustness → scale. The programme's overarching question is whether making the AI better makes the *research* better — and the three papers answer it only to the extent their experiments support.

Contents

1 The question

A language model can now write a trading strategy. Ask one for a systematic equity strategy on Indian large-caps and it will return a stated economic rationale, a feature specification, entry and exit rules, position sizing, and executable code. Ask for three hundred and it will return three hundred.

The natural next step is to backtest them, sort by Sharpe ratio, and look at the top of the list. That procedure is worthless, and it is worthless in a way that gets *worse* as the model gets better.

> The overarching question of this three-paper programme: **does making the AI better actually make the quantitative research better?**
>
> Those are two different things, and the rest of this paper exists because the literature routinely treats them as one. *Generator quality* is how well a model writes code that compiles, runs, and does what its author intended. *Research quality* is whether the resulting finding is true. A better generator raises the first by construction. Whether it raises the second is an empirical question, and the answer we measured is no.

1.1 Why the two come apart

Suppose a generator improves so much that every strategy it writes executes flawlessly on the first attempt. Nothing about that improvement makes the underlying market predictable. What it changes is the *number of plausible-looking candidates per unit of effort* — and the probability that the best of a large set looks impressive by luck alone rises with the size of the set. A better generator therefore produces a better-looking best backtest and no more real alpha. Improvement on the generator axis is improvement in the rate at which candidates arrive at the door; it is not improvement in what is behind the door.

This is not a hypothetical worry about future models. It is the result reported here, measured on three frontier models against a local 7B on the same task specification.

1.2 What this paper answers, and what it does not

AlphaAudit answers three questions and is honest about failing the fourth.

1. **Where do machine-written strategies actually die?** Not where the literature assumes. §7.
2. **Does a more capable generator change that?** Yes at the front of the funnel, not at all at the back. §8.
3. **Does a more elaborate generation *method* change it?** Not at equal compute. §9.
4. **Can an auditor identify the survivors in advance? We could not build one that does**, on any of ten corpora. §11.

The fourth is the one we set out to answer positively, and the null is reported at the same prominence as the rest. A benchmark whose reference implementation fails is still a benchmark; what would destroy it is adjusting the implementation until it passed.

2 What can go wrong with an AI-generated strategy

Before any machinery, the failure modes — in plain terms, because the rest of the paper is about detecting them and a reader who has not met them cannot judge whether the detectors are adequate.

2.1 The code reads the future

A backtest simulates trading through history. It is trivially easy, and usually accidental, to write one that uses information from after the moment it claims to be deciding. A signal computed from today's closing price cannot be used to buy at today's opening price, but nothing in ordinary Python stops you writing that. The resulting equity curve is not a slightly optimistic estimate — it is a measurement of a machine that can see tomorrow, and it can be arbitrarily good.

This is called **lookahead bias** or **leakage**. It has recognisable species:

- **Future indexing.** Directly using a value from a later timestamp — a `shift(-1)`, a reversed slice.
- **Full-sample statistics.** Computing a mean, standard deviation or quantile over the whole history and using it to make decisions early in that history. The mean of a series you have not lived through yet is not available to you.
- **Fitting before splitting.** Scaling features using statistics from the test period.
- **Snooped parameters.** A lookback window or threshold chosen because it worked on the data being tested.
- **Survivorship selection.** Choosing which stocks to trade in 2020 using the list of companies that still exist in 2025. The ones that went to zero are silently absent, and the strategy looks like genius rather than luck.

Only some of these announce themselves in the source. The last is a property of how the data was assembled, not of the strategy code at all — which is why §4 spends as much space on the universe as on the auditor.

2.2 Someone tried three hundred things

The second defect needs no bug at all. Take a market with no predictability whatsoever, generate three hundred strategies at random, and rank them. The best will look good. Not because anything was learned, but because the maximum of three hundred noisy numbers is large. This is **multiple testing**, or in this setting **backtest overfitting** (Bailey et al., 2017; Harvey et al., 2016).

The correction is not to shrink the winner's Sharpe ratio. It is to *raise the bar* it must clear, by the amount that luck alone would deliver given how many attempts were made (Bailey and López de Prado, 2014). With widely dispersed trials, two hundred attempts buy an expected best Sharpe of about 2.77 by luck — so an observed 2.0 from two hundred attempts is not impressive, it is *below* what chance produces.

That correction has a precondition which is where LLM pipelines quietly fail: **you must know how many attempts were made**, honestly, including the ones that crashed, the ones you discarded, and the ones from last week. A generator that regenerates on syntax error and reports only its successes has destroyed the denominator. §5 describes the ledger we built because of this, and §5.2 reports the one place ours is imperfect and why we did not repair it.

2.3 The costs are not a rounding error

Indian equity trading carries securities transaction tax, exchange transaction charges, an investor protection fund levy, a SEBI turnover fee, stamp duty, GST on the brokerage and exchange components, and a per-scrip depository charge on every sell. For a strategy that trades often, these compound into a drag that routinely exceeds the entire apparent edge. A great many published Indian backtests are fiction for this reason alone (Novy-Marx and Velikov, 2016; Frazzini et al., 2018).

2.4 The story and the code disagree

A machine-written strategy comes with a stated rationale. Nothing enforces that the rationale describes the code. A model can write a paragraph about liquidity provision in mid-caps and implement momentum; it can offer a mechanism so elastic that it would explain any outcome; or it can rediscover a forty-year-old anomaly and present it as novel. None of these is a bug — the code runs, the backtest is honest — and all of them mean the finding is not what it claims.

3 What was available before AlphaAudit, and why it was not enough

Four literatures each hold part of the problem, and none holds the whole of it.

Code-generation benchmarks measure whether a model writes code that passes tests (Chen et al., 2021). That is exactly the wrong success criterion here. A leaky strategy passes every test; it is *more* likely to pass, because it works better. The property we need to measure is invisible to a test suite by construction.

Backtest-overfitting statistics give us the corrections (Bailey and López de Prado, 2014; Bailey et al., 2017; Harvey et al., 2016; López de Prado, 2018) and are the reason the statistical layer in §5 is not our invention. What they do not supply is the honest denominator when the researcher is an autonomous loop. The literature assumes a human who knows roughly how many models they fitted.

Selective prediction formalises the right question — a system that declines to act when unconfident is worth more than one that is occasionally brilliant and frequently wrong (El-Yaniv and Wiener, 2010; Geifman and El-Yaniv, 2017). It has not been applied to strategy research, where the "label" arrives only after out-of-sample time has elapsed, and where abstention is nearly free while a wrong action is not.

Finance-specific language models (Wu et al., 2023) raise capability without addressing evaluation. A more capable finance model produces more plausible candidates. Section 8 is a direct measurement of what that does and does not buy.

> The gap AlphaAudit fills: there was no environment in which a machine-written strategy corpus could be scored for *validity* — survivorship-free, cost-realistic, trial-counted, holdout-disciplined — under a protocol fixed before the results were seen. Building the environment is the contribution that does not depend on our auditor working.

4 What we built

Four components, in the order a result has to pass through them. Each is a place a published Indian backtest commonly goes wrong.

4.1 A universe that includes the companies that failed

This is the most important correctness requirement in the repository, and getting it wrong invalidates everything downstream regardless of the quality of the rest.

The universe on any date t contains exactly the hundred most liquid NSE equities *as they were known on* t, including names later delisted or dropped. It is rebuilt quarterly from exchange bhavcopy archives, ranking by median traded value over a trailing window that ends strictly before the rebalance date. No constituent list from the present is ever applied backwards.

The verification that matters is not that the code looks right, it is that stocks actually leave: the suite asserts the presence of at least one symbol in an early-period universe and absent from a late one, and fails loudly otherwise. A survivorship-biased universe makes every strategy in this paper look better than it is, and the bias is invisible in every downstream number.

4.2 Costs at the rates in force on each trade's own date

`src/costs/india.py` implements the Indian statutory schedule per leg, separately for delivery and intraday, with each rate cited to its source and effective date in the docstring, and applied *as of the trade's date* rather than at today's rate. Slippage is a volume-participation model, so cost rises with order size as a share of that day's volume; a flat basis-point assumption was not acceptable and is not used.

The model is validated by hand-computing a Rs. 1,00,000 delivery round trip line by line and asserting agreement to the paisa, and independently by entering the same trade into a discount broker's public calculator. One component — the square-root market-impact coefficient — remains an uncalibrated engineering assumption, and paper 3 of this programme reports the measurement showing that daily data cannot calibrate it (Almgren et al., 2005).

4.3 An engine that raises rather than warns

Point-in-time discipline is a property of the engine, not a convention the strategy author is trusted to honour. Every signal carries the timestamp at which its information became available, and the engine raises `PointInTimeError` — an exception, not a warning — if any order's signal timestamp is not strictly prior to its fill timestamp. A signal computed from a session's close may trade no earlier than the next session's open. Cross-validation is purged with an embargo after each test fold, following López de Prado (2018), so overlapping label windows cannot leak across the split.

4.4 Fixtures: we wrote the disease before the test for it

Three strategies in `tests/fixtures/leaky/` cheat deliberately — one uses next-day returns as a feature, one fits a scaler on the full sample before splitting, one selects its universe from end-of-period membership. All three produce absurd results, which is the point: they are the positive controls for the auditor. Thirty honest strategies — moving-average crossovers, simple momentum, mean reversion — measure the false-positive rate.

What this does not say

The fixtures establish that the auditor catches cheats *of the kinds we thought to write*. They say nothing about kinds we did not. §7.1 reports a leak class on which our static layer returned a zero we do not believe.

5 The auditor

Three layers, each independently ablatable, so that the question "which layer earns its place" is answerable rather than asserted.

5.1 A_{static} — leakage by structure, not by syntax

The static layer parses candidate source with Python's `ast` module and classifies each finding by leak class, line number and severity. Its two strongest rules are structural rather than syntactic: *did the constructor accept bulk data?* and *did the decision function read stored state instead of the point-in-time view?* These mirror the engine's own information boundary, which is why they catch cheats whose results look plausible. Pattern-matching `shift(-1)` catches only the cheats that announce themselves.

Against the fixture suite the layer achieves recall 1.00 (3 of 3 leaky) and precision 1.00 (0 of 30 honest flagged).

This 100% should be discounted, and here is why

With syntax-based detectors alone, recall was 1 of 3. The structural rules were added *after* observing that failure, and were then measured on the same three fixtures that motivated them. That is closer to training accuracy than to a test, and we report it in the results rather than the appendix. The genuine out-of-sample evaluation is against generated code the detectors were never designed around (§7.1), and it is far less flattering.

5.2 A_{stat} — deflation against an honest denominator

We compute the Deflated Sharpe Ratio of Bailey and López de Prado (2014), which converts an observed Sharpe $\widehat{SR}$ into the probability that it exceeds what N trials would produce by luck, given sample length T, skewness γ_3 and kurtosis γ_4:

$$\text{DSR} \;=\; \Phi\left(\frac{(\widehat{SR} - SR_0)\sqrt{T-1}}{\sqrt{1 - \gamma_3\widehat{SR} + \frac{\gamma_4 - 1}{4}\widehat{SR}^{\,2}}}\right),$$

where SR_0 is the expected maximum Sharpe across N trials — a function of both N and the *dispersion* of the trials' Sharpes. Restating the mechanism, because it is routinely misread: deflation does not shrink the strategy's Sharpe, it raises the bar the Sharpe must clear.

The ledger. This is worthless without an honest N, and an LLM generator is a machine for losing count. We maintain a **hash-chained, append-only trial ledger outside the generator's write path**. Every draw increments it, including syntax errors, runtime failures and retries. Verification walks the chain and raises on any broken link, naming the edited entry. Nothing under `src/generate/` opens the file. Pooled across every arm in this programme the ledger stands at 3,499 entries.

One deliberate imperfection in the ledger

The local corpus's ledger holds 1,877 entries but the true draw count is 1,887, because an early run recorded one entry per candidate rather than one per draw. We did **not** retroactively edit the chain to insert the missing ten. An append-only ledger that gets rewritten when found wrong is not tamper-evident, and the property is worth more than the tidiness. The correction is applied at the point of use, the discrepancy is deliberate, and every deflation in this paper uses $N = 1{,}887$.

5.3 A_{sem} — does the story match the code?

The semantic layer presents `qwen2.5:7b-instruct-q4_K_M` with a candidate's stated rationale and its implementation and asks for one of four labels: `consistent`, `rationale_implementation_mismatch`, `unfalsifiable_mechanism`, or `unacknowledged_known_anomaly`.

Validation is against 50 items labelled by hand *before* the model was run. **Cohen's** $\kappa = 0.589$ (Cohen, 1960), raw agreement 38 of 50, at the top of the "moderate" band of Landis and Koch (1977). Per-class agreement is uneven: `consistent` 26 of 31, `unfalsifiable_mechanism` only 2 of 5. No prompt was tuned before this measurement and none was tuned after it, and the disposition — keep the layer — was fixed before the number was known.

One reconciliation, which is the ablation argument in miniature. The human reviewer labelled the deliberately-leaky `leak_full_sample_scaler` fixture as `consistent`, and that is *correct*: its rationale honestly describes what its code does, and its only defect is where the scaler was fitted. It is caught by A_{static} and correctly passed by A_{sem}. **A leaky strategy can carry a perfectly honest rationale.** The layers are not redundant, and any evaluation collapsing them into one "flagged by something" signal would obscure exactly that.

6 Experimental design

6.1 One task specification, four generators, six methods

Every arm in this programme receives the **same frozen task specification** — one prompt, whose SHA256 digest `f307433c...` is published so that any later arm can be verified as having received it verbatim. This is what makes the comparisons in §8 and §9 comparisons of the generator and the method rather than of the prompt.

Candidate i is seeded at $42 + i$; the global seed is 42; every stochastic operation takes an explicit seed. Each candidate emits a rationale, a feature specification, entry and exit rules, and executable code conforming to the engine's strategy interface. Each is evaluated in a sandboxed worker with a 180-second bound over the development window, net of costs, at a Rs. 10 lakh book. Every attempt increments the ledger.

6.2 The abstention–performance curve

For coverage $c \in (0, 1]$, rank candidates by auditor confidence, retain the top c fraction, and let $P(c)$ be the mean realised out-of-sample Sharpe of the retained set. Then

$$\text{AUAP} = \int_0^1 P(c)\,\mathrm{d}c,$$

evaluated on the grid $c \in \{0.05, 0.1, 0.2, \ldots, 1.0\}$. The baseline is rejection *at random* at matched coverage, with a bootstrap 95% interval. An auditor that carries information must

produce $P(c)$ rising as $c \to 0$: the more selective it becomes, the better what it keeps must perform on data it never saw. A flat or falling curve means the ranking is noise.

The random baseline is the whole test. Any rejection rule improves the retained set's average if the discarded candidates were worse on average — so "performance improved as we became selective" proves nothing unless it beats discarding at random by the same amount.

6.3 Two design choices that shape the result, stated plainly

Flat candidates are excluded from the ranking population rather than ranked last. They are all tied at exactly zero, and at 64% of the executed set they would dominate every retained set. Ranking them last would have been equally defensible and would have produced a different, lower curve.

Survivors handed forward to papers 2 and 3 are defined by $A_{\text{static}} \wedge A_{\text{sem}}$, excluding the statistical layer — which rejects the entire corpus and would make the handoff empty by construction. This is the one place in the programme where we chose the definition that produces a non-empty answer, and we flag it as such here rather than in a footnote.

6.4 Holdout discipline

The 2025 calendar year was frozen as a holdout before any holdout data was fetched, set to a calendar boundary rather than to the latest available session so that window length could not become a researcher degree of freedom.

It has been evaluated three times for paper 1, each under written authorisation logged with a timestamp: once on a gross-of-costs pipeline, once after that pipeline was found defective and rebuilt, and once after the ranking defect described in §11.1. The first evaluation is retired and its artefacts frozen under a directory whose README forbids quoting them without the word "gross"; the second is superseded and retained beside the third.

Every one of the three was authorised in advance to repair a defect, and no parameter, threshold, prompt or model was changed after any of them. That is the property that matters, and it is a different property from the count: a holdout is contaminated by iterative optimisation against it, not by correct arithmetic being performed on it more than once. We recognise that "we re-ran it and the conclusion held" is what a researcher would say in both the honest and the dishonest case, so we publish the full before-and-after of each correction and the diffs that produced them rather than asking to be believed. **The paper-1 holdout is permanently closed.**

The frontier and generation-method arms of §8 and §9 were evaluated on the same holdout once each, under a separate written authorisation, on an apparatus frozen beforehand and unmodified since — reuse being contaminating when one method is repeatedly tuned against the same data, whereas here each generator is a new subject scored by a frozen instrument.

7 The local-model experiment: where strategies actually die

> **[LOCAL 7B] The dominant failure mode of an LLM strategy researcher is not that it cheats. It is that it does not run.**

One local model (`qwen2.5:7b-instruct-q4_K_M`, temperature 0.8), one prompt, 1,550 draws in two batches pooled only after asserting identical model tag, prompt digest, seed rule and window.

Takeaway. Read the second row. *Every* draw returned syntactically valid Python defining a conforming strategy with a rationale. The model is entirely reliable at producing code that looks right. All the attrition happens on

Table 1. The funnel. $n = 1,550$ draws, one model, one prompt.

Stage	Count	Share
Draws requested	1,550	—
Returned parseable, interface-conforming code	**1,550**	**100.0% of draws**
Executed to completion against real data	**631**	40.7% of corpus
— of those, flat: ran 1,232 sessions, never traded	406	64.3% of executed
— of those, rankable: executed *and* traded	**225**	**14.5% of corpus**
Runtime error	894	57.7%
Timeout at 180 s	25	1.6%

contact with data.

The 894 runtime errors are dominated by a single confusion. `AttributeError` (327) and `ColumnNotFoundError` (247) together account for 64% of failures, and both trace overwhelmingly to the model calling pandas methods on polars frames, or treating a wide frame as though it were long. The prompt states the distinction explicitly and carries a worked example. The model makes the error anyway, in roughly a fifth of all draws.

> This relocates the question the project was built to ask. "What fraction of LLM-generated strategies fail static leakage screening" presumes leakage is the binding constraint. On this corpus it is not: the auditor was never reached for 59% of the corpus, and for a further 26% it had nothing to deflate. **An auditor tuned to catch cheating is guarding a door most candidates never reach.**

7.1 Leak classes, and one number we do not believe

Static analysis needs no execution, so it ran over all 1,550 sources: 222 rejected (14.3%), of which 195 carried one class, 26 two, and one three.

`snooped_parameter`	166	`boundary_crossing_window`	2
`future_indexing`	58	`survivorship_selection`	1
`target_in_features`	23	`full_sample_statistic`	**0**

We do not believe the zero

Full-sample statistics are the most common leak class in hand-written quantitative code, and a corpus of 1,550 candidates containing none of it is not plausible. The cause is almost certainly ours: to raise the executable rate from near zero, the prompt advises working in plain Python rather than in long polars expression chains, and `pl.col("x").mean()` over an unfiltered frame is precisely where this class arises. The distribution above therefore describes **this prompt**, not LLM-generated strategies in general, and the zero should be read as evidence that the instrument is blind here — not that the defect is absent.

Most leak flags land on code that never produced a result. Static analysis is a statement about source, not about realised profit, and the two are correctly independent: a leak is a leak whether or not the strategy went on to trade. But *where* the flags land is itself a measurement.

Fate of the 222 static-rejected candidates	Count	Share
Runtime error — never ran	114	51.4%
Evaluated but flat — ran, never traded	79	35.6%
Timeout	1	0.5%
Evaluated and traded	**28**	**12.4%**

Only 28 leak findings in the entire corpus attach to a strategy that took a position. A worked example makes the mechanism concrete. Candidate 053 is flagged `future_indexing` at confidence 1.0 for a `shift(-1)` whose result is then filtered onto a past session date — textbook future indexing, correctly caught. But the block is unreachable: the guard preceding it requires 21 rows from a view that returns at most 20, so it is false on every session and the function exits before the leak executes. Had it been reached it would have raised twice over.

The verdict and the flat return series are compatible, not contradictory. What the pairing shows is that **a leakage-prevalence rate measured over generated source is not a deployment-risk rate**. Reported as "14.3% of candidates leak", the static layer sounds like it is intercepting a substantial hazard. Reported as "28 of 1,550 candidates leak *and* trade", the same measurement says the hazard is rare because the population is inert. Both numbers are true. Only the second is decision-relevant, and we are not aware of prior work that separates them.

The semantic layer labelled 90.1% of the corpus `consistent`, rejecting 154 (9.9%): 119 rationale–implementation mismatches, 21 unacknowledged known anomalies, 14 unfalsifiable mechanisms. It is not a detector that fires indiscriminately. Whether the 154 are the *right* ones is not established on this population, since κ was measured on a different one.

8 The frontier-model comparison

> **[FRONTIER] Frontier models solved the execution problem. They did not solve the evidence problem.**

The obvious objection to §7 — and the first one a referee reaches for — is that we measured a small local model, not AI strategy generation. So the identical frozen task specification was issued to three frontier models, and every candidate was run through the identical unchanged funnel. **This is not an appendix.** The generator is part of the scientific design, because the programme's question is what changes when the AI gets better.

4 requests per arm, 60 candidates in total, 20 per arm.

Takeaway. Capability moves the left panel to its ceiling and does not move the right panel in the direction that would matter.

8.1 The front of the funnel: solved

Every frontier arm executed and traded at 100%. The execution barrier that consumes 85% of the local corpus is not a property of machine-written strategies; it is a property of a 7B model, and it vanishes with capability. Total static leak findings across all 60 frontier candidates: 1. Frontier models do not write the cheats our static layer was built to detect — but note that the local corpus barely did either (28 of 1,550 that both leaked and traded), so this is a small effect measured on a small sample, not a demonstration that capability eliminates leakage.

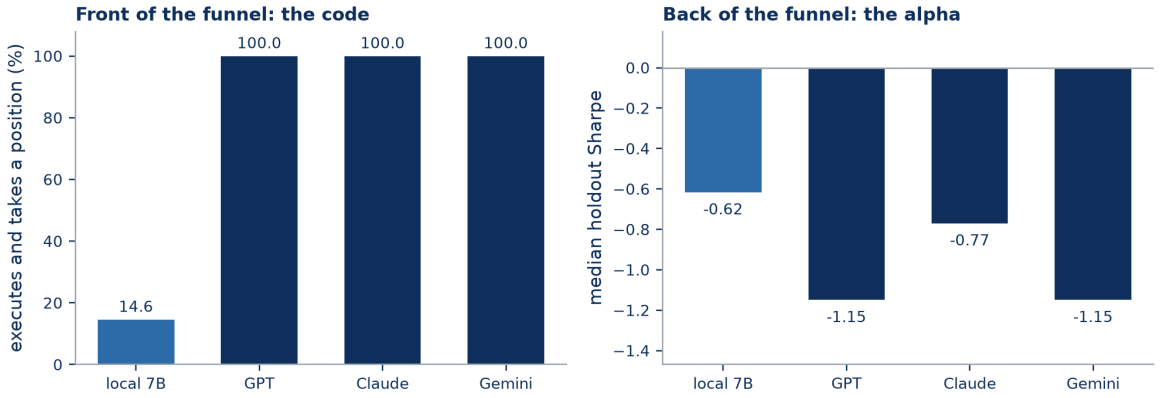

Figure 1. The same generators on one axis, measured at two points in the funnel. Left: the fraction of draws producing code that runs on real data and takes a position. Right: median Sharpe on the never-touched holdout year. Sample sizes differ by up to 40× between the reference and the frontier arms; see the scope box below.

8.2 The back of the funnel: unmoved

[FRONTIER] Zero of 60 frontier candidates cleared the significance bar at their own honest trial counts, on development or on holdout. The local corpus cleared 0 of 631. Capability changed the numerator of the funnel by a factor of seven and left this at zero.

Every frontier arm's median holdout Sharpe is *worse* than the local reference's, and every arm's mean is negative on both windows. The best single holdout Sharpe in any frontier arm is +0.4419 — positive, but nowhere near clearing deflation at that arm's own trial count, and selected as the maximum of twenty.

What this does not say

The supportable claim is that frontier models did not *improve* out-of-sample performance — not that they produced none. Three things block the stronger claim.

- Median holdout Sharpe is worse for all three arms, but the *fraction* with a positive holdout Sharpe is better: 40%, 35% and 30% against the local arm's 27.6%. Those two facts point in opposite directions and we report both.
- The populations are differently selected. The local arm's 27.6% is measured on the 14.6% of draws that survived execution; the frontier arms' on 100% of draws. The local survivors passed a filter the frontier candidates never faced.
- **The generator axis is not compute-matched.** 20 frontier draws against 830 local draws, at unequal and unmeasured token cost. Under Rule 11 of this programme's charter no generator-axis comparison may be reported as an efficiency win, and none is.

8.3 The matched-size control, and why small samples flatter

Comparing a 20-draw arm against a 1,550-draw corpus is not a fair deflation comparison, because deflation punishes trials: a 20-trial arm faces a much lower bar than a 1,550-trial one. So we drew 1,000 random subsamples of the local corpus at each frontier arm's size and deflated

Table 2. Generator axis. Identical prompt, identical funnel, **not compute-matched** — these are rate claims only, never efficiency claims.

Quantity	local 7B	GPT	Claude	Gemini
Draws	830	20	20	20
Executed on real data	40.7%	100.0%	100.0%	100.0%
Executed *and* traded	14.6%	100.0%	100.0%	100.0%
Static leak findings	—	0	0	1
Semantic rejections	—	2	6	5
Statistical rejections	**all**	20	20	20
Mean Sharpe, development	—	-0.1394	-0.1635	-0.5854
Mean Sharpe, holdout	—	-0.9491	-0.5781	-1.4768
Median Sharpe, holdout	-0.616	-1.149	-0.770	-1.149
Best Sharpe, holdout	—	+0.3376	+0.4419	+0.3827
Probability of backtest overfitting	—	0.0190	0.0378	0.0037
Mean Sharpe lost to costs	0.5867	0.5004	0.5795	0.8120
Sharpe sign reversed by costs	35/225	6/20	1/20	4/20
Cleared DSR ≥ 0.95, development	**0/631**		**0 of 60**	
Cleared DSR ≥ 0.95, holdout	**0/631**		**0 of 60**	

each subsample at its own trial count, recomputing trial dispersion *within* the subsample — because that is the only information a 20-draw arm actually has.

Subsamples of the local corpus clearing DSR ≥ 0.95	at $n = 5$	at $n = 20$
Development window	41/1,000	3/1,000
Holdout	**0/1,000**	**0/1,000**

Takeaway. On development, a five-draw slice of the *local* corpus clears the bar 4.1% of the time — purely because five trials are cheap to deflate. No frontier arm cleared it at twenty. On the holdout, nothing clears at any size: 0 of 1,000 local subsamples, and 0 of 60 frontier candidates. This is the control that shows the frontier arms' failure is not a small-sample artefact.

8.4 Two vendors wrote the same strategy, and it was the median

[FRONTIER] **[EXPLORATORY]** Three GPT candidates and two Gemini candidates produce holdout return series **identical to within** 8.7×10^{-19} **across all 245 sessions** — the same turnover, the same volatility, the same return. Two different vendors, given the same prompt, converged on behaviourally identical code. **That cluster sits exactly on the median of both arms**, which is why GPT and Gemini report the same median holdout Sharpe to sixteen decimal places (-1.149 and -1.149).

This was found while checking whether the identical medians were a bug. They are not; they are a measurement. Duplicate clustering is not a local-model pathology — it recurs at the frontier, across vendors. It also means a per-arm mean or median can be a statement about one duplicated idea rather than about twenty independent ones, and any comparison across these arms inherits that.

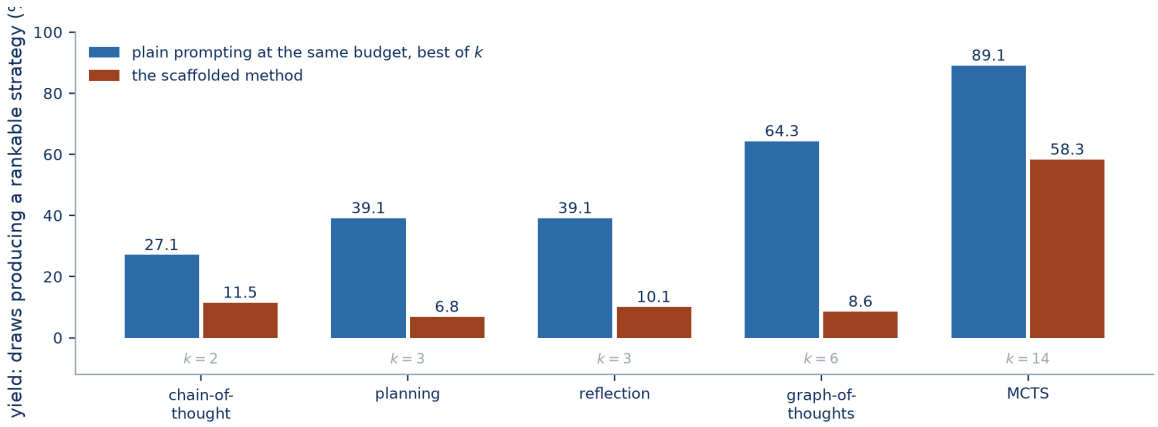

Figure 2. Each scaffolded method against plain prompting at the same measured token budget. k is the number of plain-prompting attempts the scaffolded method's token spend buys.

What this does not say

This is an **exploratory** finding: it was not pre-registered, it was discovered while verifying another number, and it rests on five candidates. It is reported because it materially affects how the medians in Table 2 should be read, not because five candidates establish a general property of frontier models.

9 Advanced generation methods, at equal compute

[GENERATION METHOD] [PRE-REGISTERED] Of 5 scaffolded generation methods, 0 beat plain prompting once plain prompting was given the same token budget.

If capability does not help, perhaps method does. The same local model, the same frozen task specification, and 6 generation methods: plain prompting, chain-of-thought (Wei et al., 2022), planning, reflection (Shinn et al., 2023), graph-of-thoughts (Besta et al., 2024), and Monte Carlo tree search over candidate programs (Yao et al., 2023). 2,495 draws in total.

9.1 Why the naive comparison is worthless

A five-agent committee that spends five times the tokens of a single prompt and produces better output has demonstrated nothing. Of course it does — it thought five times as hard. The honest control is not plain prompting at its natural budget; it is **plain prompting run k times at the same total token budget, allowed to keep its best**. We compute k from measured tokens per accepted strategy, and we count barren blocks in the denominator so that a method is not rewarded for the attempts it wasted.

Takeaway. Every scaffolded method loses to its own control. The gap widens with the amount of scaffolding: graph-of-thoughts spends $k = 6$ plain prompts' worth of tokens per strategy, and tree search spends $k = 14$.

9.2 The one arm that looks like a win, and why we do not report it as one

Tree search reaches 58.3% yield and 43.3% full-stack survival against plain prompting's 14.6% and 8.6% — by far the highest in the programme. It is also the arm whose compute-matched control is highest of all, at 89.1% against its own 58.3%.

Table 3. Methodology axis, all on the same local model at matched token budget. Yield is the fraction of draws producing a rankable strategy. *Every* arm loses to its control.

Method	Draws	k	Control yield	Method yield	Redundancy	Holdout med.
plain prompting	830	—	—	14.6%	13.0%	-0.616
chain-of-thought	800	2	27.1%	11.5%	17.3%	-0.051
planning	365	3	39.1%	6.8%	0.0%	-0.060
reflection	288	3	39.1%	10.1%	8.0%	-1.151
graph-of-thoughts	152	6	64.3%	8.6%	0.0%	-0.507
tree search	60	14	89.1%	58.3%	14.3%	0.047

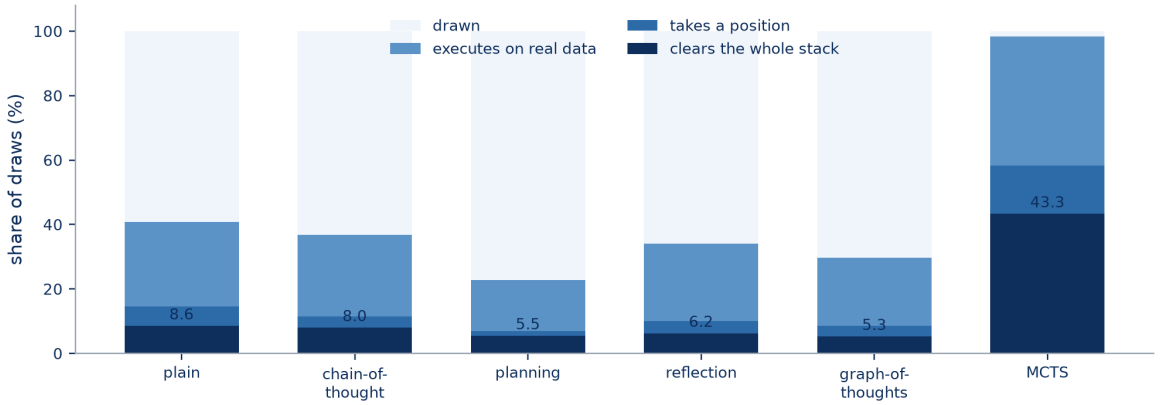

Figure 3. Where each method's corpus dies, as a share of its own draws. The dark bar is the share clearing the entire evaluation stack.

What this does not say

Tree search is treated as an anomaly to investigate, not an effect to report, on the principal investigator's ruling and for three reasons stated before its downstream numbers were examined. Its sample is the smallest in the programme (60 draws). Its k is the largest (14), so its control is correspondingly generous. And a jump of this size in this domain is, on the programme's own standing rule, more likely to be a defect than a discovery. We did not delete the arm — deleting an inconvenient arm is worse than reporting a confusing one — and we do not claim it as a win.

9.3 Does scaffolding trade diversity for quality?

The hypothesis stated in advance was that heavier scaffolding induces mode collapse: quality rises while diversity falls. The measurement is redundancy — the share of a method's tradeable output sitting inside a cluster of behaviourally identical return series, detected by union-find over realised returns.

Redundancy does not rise with scaffolding: chain-of-thought is highest at 17.3%, and planning and graph-of-thoughts report 0.0% and 0.0%. But those two zeros are **not evidence of diversity**, and this is a case where reporting the number without its power would mislead. At the reference duplicate rate observed in the two large arms, a sample of planning's size would return zero duplicates by chance 61.2% of the time, and one of graph-of-thoughts' size 88.0% of the time. **Those zeros are consistent with the same duplication rate as everything else.** Only the two large arms have the power to say anything: there, zero duplicates would

Table 4. Gross to net over the development window, local corpus. $n = 225$ traded candidates. Gross and net come from a *single run* — the engine records the pre-cost series alongside the post-cost one — so the drag is a measured quantity on one path, not a difference between two runs.

Quantity	Value	Sample
Best Sharpe, gross	1.3049	225
Best Sharpe, net	**1.2807**	225
Mean Sharpe reduction	0.5867	225
Median Sharpe reduction	**0.1943**	225
Mean turnover, annualised	82.2×	225
Median turnover, annualised	17.6×	225
Mean cost drag, annualised	58.13%	225
Median cost drag, annualised	4.39%	225
Sharpe sign reversed, $+\to-$	**35 (15.6%)**	225
Sharpe sign reversed, $-\to+$	**0**	225
Ruined outright (equity reached zero)	11	631

occur 0.0% and 0.2% of the time, and duplicates were observed.
Takeaway. We cannot confirm or reject the mode-collapse hypothesis on the small arms. Saying so is the result; quoting the zeros as diversity would not be.

Cross-arm duplication is the sharper measurement, since it does not depend on within-arm sample size: 13.9% of plain prompting's tradeable output and 19.8% of chain-of-thought's is behaviourally identical to something another arm produced. Different methods, same strategies.

10 What survives costs, multiple testing and the holdout

10.1 Costs are a selection mechanism

Mean and median diverge by a factor of thirteen on cost drag. The mean is dragged by a tail churning above 80× annual turnover; the median strategy loses 0.19 of Sharpe and 4.4% a year. Both are reported because quoting either alone misleads.

The best gross strategy is also the best net strategy — because it barely trades, at 2.8× annual turnover and 0.4% annual cost. **Cost survival selects for low turnover, not for skill**, the same conclusion Novy-Marx and Velikov (2016) reach for US anomalies, arrived at here through a very different door.

Standard factors are not spared. Eleven textbook factors through the same pipeline as a positive control: drag rises monotonically with turnover in ten of eleven rows, and two factors flip from profitable to unprofitable. Momentum (Jegadeesh and Titman, 1993) survives at a net Sharpe of 0.9656 — but equal-weighting the same universe returns 0.9615. **Net of Indian costs, the momentum premium on this universe is inside the noise.**

The eleventh row, and why the assumed book size decides the verdict. The one factor breaking monotonicity is inverse-volatility weighting: the *lowest* turnover of any active strategy (0.011 per session) and the second *highest* cost (10.18 bps per session). It holds a hundred names and nudges each daily, and the depository charge is levied per scrip regardless of size.
Takeaway. The depository contribution falls 169× across a 100× book sweep, close to the inverse proportionality the model predicts, and **the strategy's verdict reverses with assumed capital**: −1.38 at Rs. 10 lakh, +0.69

Table 5. Depository-charge contribution by book size, obtained by differencing against a no-DP ablation rather than recomputed from the rate.

Strategy	Book	Total bps/day	DP bps/day	DP share	Sharpe
inverse_volatility	Rs. 10 L	10.183	9.963	97.8%	−1.3792
inverse_volatility	Rs. 1 Cr	0.866	0.597	69.0%	+0.6359
inverse_volatility	Rs. 10 Cr	0.590	0.059	10.0%	+0.6922
momentum_skip_month	Rs. 10 L	1.685	0.066	3.9%	0.9656
momentum_skip_month	Rs. 10 Cr	3.944	0.001	0.0%	0.7506

at Rs. 10 crore. Momentum runs the opposite way. Small books are killed by flat fees, large books by market impact, and the two regimes cross over.

We report the corpus at Rs. 10 lakh, which was the pre-existing default, and we did not change it after observing that it flips a sign.

10.2 Nothing clears deflation, on any arm

At an honest trial count of $N = 1{,}887$, **zero of 631 executed local candidates attain a deflated Sharpe probability above 0.95**; the corpus maximum is 4.4×10^{-8}. Zero of 60 frontier candidates clear it at their own trial counts. The best frontier deflated probability in any arm is 4.5×10^{-39}.

What this does not say

We cannot separate "deflation is the right bar and nothing here is real" from "deflation at this trial count is unclearable by construction on this corpus". The statistical layer rejects 100% of every population it has been shown, which means it has never been observed to discriminate. A layer that rejects everything is either correct about a worthless corpus or non-informative under this evaluation regime, and this experiment cannot tell which. The principal investigator's ruling was to leave the question open rather than resolve it by choosing the flattering reading. It is the largest open question in the programme.

11 The auditor null

[GENERATOR-INDEPENDENT] Across 10 independently generated corpora and all seven combinations of the three auditor layers, **0 of 70 configurations beat random rejection at matched coverage** on out-of-sample data.

Takeaway. The result does not depend on the generator, the generation method, or the corpus. Every dot is inside or below its interval.

Six of the ten corpora place the best configuration *below* the random interval — worse than discarding at random. The pooled local corpus's best configuration is -1.1967 against a random interval of $[-1.2208, -0.8600]$; the best frontier arm is -0.6083 against $[-0.8711, -0.3004]$. Across the frontier arms, 0 of 21 configurations beat random.

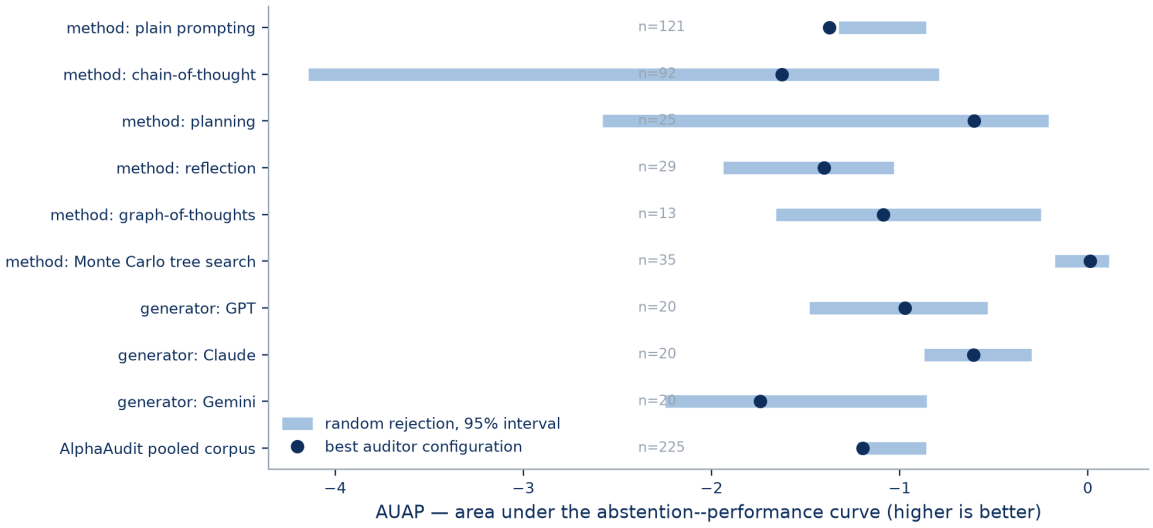

Figure 4. One row per corpus. The bar is that corpus's own random-rejection 95% interval; the dot is the best of seven auditor configurations. A working auditor's dot sits to the right of its bar. None does. Sample sizes are printed per row and range from 13 to 225.

The configuration that beat random in sample was the one that provably cannot. Its ranking signal is a monotone function of the quantity it was scored against, so its in-sample advantage is arithmetic rather than information. Out of sample it fails with the rest.

11.1 Two corrections, and what they did to the conclusion

We found two defects in our own pipeline after first computing this null

The first was the entire cost implementation: the initial null was computed on a gross-of-costs pipeline, which was found defective and rebuilt. The second was a sign error in the ranking itself, which had been inverting the auditor's confidence order.

Both were repaired, the holdout was re-opened once for each under written prior authorisation, and the null was recomputed. **The magnitudes moved. The conclusion did not.** No parameter, threshold, prompt or model was changed after any holdout evaluation, and the diffs that produced each correction are published rather than described.

We report this because a reader is entitled to ask whether a null that survived two corrections of its own apparatus is a null or an artefact of a third defect not yet found. We cannot rule that out. What we can do is publish the apparatus.

11.2 What the null does and does not establish

What this does not say

The null is confounded with corpus degeneracy and cannot separate "the auditor is uninformative" from "these corpora are too weak to test it".

The population being ranked is one in which 14.6% of draws trade at all, nothing clears deflation, and 15.6% of the profitable reverse sign under costs. Asking an auditor to rank that population for out-of-sample quality may be asking it to order noise. A stronger generator would test the auditor properly — and the frontier arms, which are a stronger generator, produce the same null on a population that executes at 100%. That narrows

the confound. It does not eliminate it, because those arms are 20 candidates each.

12 What the auditor can and cannot detect

Setting aside whether the ranking works, the layers have measurable and asymmetric competence.

Layer	Detects, with evidence	Blind to, with evidence
A_{static}	All 3 leaky fixtures; 0 of 30 honest ones flagged; 4 leak classes observed in the wild	`full_sample_statistic`: **0** findings in 1,550 sources, which we do not believe (§7.1). Unreachable code: flags a leak that never executes.
A_{stat}	Trial inflation, with a tamper-evident denominator; recovers the known result that a Sharpe of 2.0 from 200 dispersed trials is below chance	Discrimination. It has rejected 100% of every population shown to it, so it has never been observed to separate anything from anything.
A_{sem}	Rationale–implementation mismatch at $\kappa = 0.589$ on 50 pre-labelled items; correctly passes a leaky fixture whose rationale is honest	`unfalsifiable_mechanism`: 2 of 5 agreement. Whether its 154 rejections on the corpus are the right ones is not established, since κ was measured on a different population.

Takeaway. The layers are not redundant — the fixture that A_{static} catches is the one A_{sem} correctly passes — but neither is their union sufficient, and the statistical layer's behaviour is currently indistinguishable from a constant.

13 Threats to validity

The prompt is a confound we cannot remove. The prompt advises plain Python over polars expression chains to raise executability, and that advice plausibly suppresses the `full_sample_statistic` class to zero. Every leak-class distribution in this paper describes this prompt.

One market, one period. Indian equities, 2020–2024 development, 2025 holdout, the hundred most liquid NSE names. Cost structure, retail participation and liquidity are specific to that market; the execution-failure rates are specific to the libraries in the prompt.

Sample sizes differ by up to 40× across arms. 20 frontier draws against 830 for plain prompting; 60 for tree search against 830. Where a comparison is underpowered we compute the power and say so (§9); we do not average across arms of different size, and no table in this paper pools them.

The generator axis is not compute-matched, by construction. Token spend for hosted models is not comparable to local inference, so §8 reports rates only. No sentence in it claims a frontier model is more *efficient*.

The static layer's fixture score is closer to training accuracy than to a test. Stated in §5 where the number appears, not here.

The statistical layer's 100% rejection rate is unresolved. Stated in §10. It is the threat we are least able to bound.

Numbers predating the macro pipeline are stated inline. The local-corpus cost and leak-class tables were computed before the generated-macro discipline was adopted, and are transcribed rather than macro-substituted. They are regenerable from `data/raw/` by documented commands, but they are not machine-inserted into this document, and that is a real gap in the paper's own reproducibility guarantee.

Impact remains an assumption. The square-root market-impact coefficient in the cost model is uncalibrated. Paper 3 reports the measurement showing daily data cannot calibrate it, so this is a permanent property of the apparatus rather than a to-do.

14 What this tells us about AI-driven research

> **Can AI write code? Yes, increasingly and now essentially completely. Can AI generate statistically credible quantitative research? On this apparatus, much less clearly — and the first capability improving did not cause the second to.**

Three claims, at three different scopes, which the rest of this section keeps separate because collapsing them is the error the programme was built to avoid.

Generator-independent [GENERATOR-INDEPENDENT] The auditor null holds on all 10 corpora, produced by four generators and six methods. Nothing clears deflation on any arm. Costs reverse signs on every arm. These are the claims that survive a change of generator, and they are the ones that license the apparatus being used by anyone else.

Generator-specific [LOCAL 7B] [FRONTIER] The execution barrier is a property of the local 7B, not of machine-written strategies: 14.6% against 100%. Any paper reporting execution rates for one model is reporting that model. Our own paper 1 result, published before the frontier arms existed, was such a claim, and this section is where we say so.

Method-specific [GENERATION METHOD] Scaffolding buys candidates, not quality, at equal budget: 0 of 5 arms beat their control. This is the claim most likely to generalise beyond finance, because it is a claim about compute accounting rather than about markets.

14.1 The one recommendation we are willing to make

If you evaluate a machine-written quantitative finding, the ordering that matters is: *does it run*, then *does it survive costs*, then *does it survive the number of things you tried*, then *does it survive data you have never seen*. Most published pipelines measure the first and report the fourth. The attrition between them, on this corpus, is total.

15 The AlphaAudit benchmark

Frozen and released at github.com/Ashwin-aadi/Trivijaya-Quant-Research:

- **Environment.** Survivorship-free point-in-time universe, verified cost model, raising point-in-time engine, purged cross-validation with embargo.
- **Fixtures.** 3 deliberately-leaky and 30 honest strategies, permanent assets.
- **Corpus.** 2,495 machine-written candidates across 6 generation methods, plus 60 frontier candidates and the 1,550-draw local corpus, each with audit verdicts, backtests, and holdout results where applicable.
- **Protocol.** The frozen task specification and its digest, the abstention–performance harness, the random-rejection baseline, and the trial ledger with its verification.
- **Every number.** `RESULTS.md` per benchmark, and the generated macro files this paper is built from. If this paper and the repository disagree, the repository is right.

A stronger auditor can be scored against exactly this apparatus and contest the null directly. That is the point of releasing an environment whose reference implementation failed.

16 Conclusion

We built an environment in which a machine-written trading strategy can be scored for validity rather than for plausibility, and we ran four generators and six generation methods through it without changing it between arms.

The local model's strategies fail as programs, not as research: 100% produce conforming code and 14.6% produce a strategy that trades. Frontier models remove that barrier entirely, at 100%. **They remove nothing else.** Zero of 60 frontier candidates clear deflation; 11 reverse sign under costs; every arm's median holdout Sharpe is negative. More elaborate generation methods lose to plain prompting at every matched token budget we measured. And across 10 corpora, 0 of 70 auditor configurations beat random rejection out of sample.

> **Better code did not mean better quantitative research.** That is one measurement, on one market, on one apparatus, with the confounds named in §13. It is also the measurement the field is currently not making, and the apparatus for making it is now public.

Where this leaves us

This paper asked whether a machine-written strategy is *real*. For most of the corpus the answer was no, and the survivors it hands forward — defined by $A_{\text{static}} \wedge A_{\text{sem}}$, since the statistical layer leaves nothing — are a small set of low-turnover strategies that survived costs and one holdout year.

Surviving one history is a weak claim. The 2020–2024 development window contains one pandemic crash, one full rate cycle and one structural liquidity shift; a strategy that survived it survived a single sequence of events, and the number of independent regime observations available is very small. **The next question is therefore not whether these strategies are real, but when they break** — and answering it requires plausible histories that did not happen, because history did not supply enough of them.

That is paper 2, **RegimeStress**.

References

Almgren, R., Thum, C., Hauptmann, E., and Li, H. (2005). Direct estimation of equity market impact. *Risk*, 18(7), 58–62.

Bailey, D. H., and López de Prado, M. (2014). The Deflated Sharpe Ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5), 94–107.

Bailey, D. H., Borwein, J., López de Prado, M., and Zhu, Q. J. (2017). The Probability of Backtest Overfitting. *Journal of Computational Finance*, 20(4), 39–69.

Besta, M., Blach, N., Kubicek, A., et al. (2024). Graph of Thoughts: Solving Elaborate Problems with Large Language Models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16), 17682–17690.

Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*.

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46.

El-Yaniv, R., and Wiener, Y. (2010). On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11, 1605–1641.

Frazzini, A., Israel, R., and Moskowitz, T. J. (2018). Trading costs. *SSRN Working Paper 3229719*.

Geifman, Y., and El-Yaniv, R. (2017). Selective classification for deep neural networks. *Advances in Neural Information Processing Systems*, 30.

Harvey, C. R., Liu, Y., and Zhu, H. (2016). . . . and the Cross-Section of Expected Returns. *The Review of Financial Studies*, 29(1), 5–68.

Jegadeesh, N., and Titman, S. (1993). Returns to buying winners and selling losers: Implications for stock market efficiency. *The Journal of Finance*, 48(1), 65–91.

Landis, J. R., and Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 159–174.

López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley.

Novy-Marx, R., and Velikov, M. (2016). A taxonomy of anomalies and their trading costs. *The Review of Financial Studies*, 29(1), 104–147.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems*, 36.

Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.

Wu, S., Irsoy, O., Lu, S., et al. (2023). BloombergGPT: A Large Language Model for Finance. *arXiv:2303.17564*.

Yao, S., Yu, D., Zhao, J., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *Advances in Neural Information Processing Systems*, 36.